

1. Structura unui compilator - definire pe scurt

Intrebare: Ce este un compilator si care este structura lui generala?

Raspuns:

Un **compilator** traduce un program scris intr-un limbaj de nivel inalt, cum ar fi FORTRAN, PASCAL, C, C++ sau JAVA, intr-un limbaj masina. Compilatorul primeste la intrare un **program sursa** si furnizeaza la iesire un **program obiect echivalent** si un fisier de erori.

Traducerea programului sursa in program obiect se face in mai multe faze:

1. **Analiza lexicala** - grupeaza caracterele programului in subsiruri numite **atomi lexicali**: cuvinte cheie, operatori, constante, identificatori. Spatiile care separa atomii lexicali sunt eliminate.
2. **Analiza sintactica** - preia sirul de atomi lexicali si depisteaza structuri sintactice precum expresii, liste, instructiuni si proceduri, plasandu-le intr-un arbore sintactic.
3. **Analiza semantica** - imbogateste arborele sintactic prin informatii suplimentare introduse in tabela de simboluri si conduce generarea codului intermediar. O componenta importanta este verificarea tipurilor.
4. **Generarea codului intermediar** - produce un sir de instructiuni simple, cu format fix. Operatiile sunt apropiate de cele ale calculatorului, iar ordinea instructiunilor respecta ordinea executiei.
5. **Optimizarea codului** - elimina redundantele, calculele inutile si variabilele inutile pentru a obtine o executie mai eficienta.
6. **Generarea codului obiect** - selecteaza locatiile de memorie si translateaza codul intermediar intr-o secventa de instructiuni masina.
7. **Gestiunea tabelii de simboluri** - inregistreaza identificatorii folositi in programul sursa si colecteaza attributele lor: tip, memorie alocata, parametri, mod de transmitere etc.
8. **Tratarea erorilor** - include proceduri activate cand este depistata o greseala. Utilizatorul primeste un mesaj de eroare, iar compilarea trebuie sa continue cel putin pentru detectarea altor erori.

Sursa in curs: Curs 6, II.1.1. Structura unui compilator, pp. 81-85.

2. Gramatici - definitie, derivare, limbaj generat, clasificarea Chomsky

Intrebare: Ce este o gramatica formală, ce înseamnă derivare și ce este limbajul generat de o gramatică?

Raspuns:

O **gramatică** este un sistem:

$$G = (N, \Sigma, P, S)$$

unde:

- N este alfabetul neterminalelor; elementele sale se notează de obicei cu litere mari;
- Σ este alfabetul terminalelor; elementele sale se notează de obicei cu litere mici;
- P este mulțimea regulilor de producție;
- $S \in N$ este simbolul inițial.

O producție se notează de forma:

$$u \rightarrow v$$

și exprimă faptul că, într-un cuvânt, subcuvântul u poate fi înlocuit cu subcuvântul v . În fiecare producție, cuvântul din partea stângă trebuie să conțină cel puțin o variabilă/neterminale.

Derivare directă

Pe mulțimea cuvintelor se definește relația $\Rightarrow$. Scriem:

$$w \Rightarrow \omega$$

și citim „ w derivă direct ω ” dacă există o producție $u \rightarrow v$ și cuvinte α, β , astfel încât:

- $w = \alpha u \beta$
- $\omega = \alpha v \beta$

Adică ω se obține din w prin înlocuirea unei apariții a lui u cu v .

Derivare în mai mulți pași

Inchiderea reflexivă și tranzitivă a relației $\Rightarrow$ se notează cu $\Rightarrow^*$. Aceasta permite derivări în zero, unul sau mai mulți pași. Derivarea în k pași se notează cu $\Rightarrow^k$.

Limbaj generat

Limbajul generat de gramatică G este mulțimea cuvintelor terminale care pot fi obținute din simbolul inițial:

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$$

Doua gramatici care genereaza acelasi limbaj se numesc **echivalente**.

Clasificarea Chomsky

Chomsky a introdus o clasificare a gramaticilor in functie de restrictiile impuse productiilor:

1. **Gramatici de tip 0** - nu au restrictii impuse asupra productiilor.
2. **Gramatici de tip 1 / dependente de context** - orice productie $u \rightarrow v$ satisface conditia $|u| \leq |v|$.
3. **Gramatici de tip 2 / independente de context (GIC)** - orice productie are in stanga un singur neterminal: $u \in N$, $|u| = 1$, iar partea dreapta nu este vida.
4. **Gramatici de tip 3 / regulate** - partea stanga este un neterminal, iar partea dreapta are forma specifica gramaticilor regulate; in forma redusa, productiile au partea dreapta de forma terminal sau terminal urmat de neterminal.

Daca L_i este familia limbajelor generate de gramaticile de tip i , atunci:

$$L_3 \subset L_2 \subset L_1 \subset L_0$$

Sursa in curs: Curs 1, I.1. Clasificarea gramaticilor dupa Chomsky, pp. 9-12.

3. Recursivitatea limbajelor - definitie

Intrebare: Cand este un limbaj recursiv?

Raspuns:

Un limbaj L peste alfabetul Σ este **recursiv** daca exista un algoritm care, pentru orice secventa $w \in \Sigma^*$, determina daca $w \in L$ sau $w \notin L$.

Cu alte cuvinte, problema apartenentei unui cuvant la limbaj este rezolvabila algoritmic: algoritmul se opreste si raspunde „DA” sau „NU” pentru orice cuvant de intrare.

In curs este precizat ca **limbajele de tip 1 sunt recursive**.

Sursa in curs: Curs 1, sectiunea despre testarea recursivitatii, p. 12.

4. Automate finite - definitie, configuratie, trecere, limbaj acceptat

Intrebare: Ce este un automat finit si cum accepta un limbaj?

Raspuns:

Un **automat finit** are doua componente principale:

- o **banda de intrare**, infinita la dreapta, impartita in celule;
- o **unitate centrala**, care se poate afla intr-un numar finit de stari.

In functie de starea curenta si de simbolul citit de pe banda de intrare, unitatea centrala isi poate schimba starea.

Un automat finit este un sistem:

$$A = (Q, \Sigma, \delta, q_0, F)$$

unde:

- Q este multimea starilor automatului;
- Σ este alfabetul de intrare;
- $\delta : Q \times \Sigma \rightarrow P(Q)$ este functia de tranzitie;
- q_0 este starea initiala;
- F este multimea starilor finale.

Configuratie

O **configuratie** a automatului este o pereche:

$$(q, \alpha)$$

unde:

- q este starea curenta;
- α este secventa de pe banda de intrare ramasa de citit.

Configuratia initiala este:

$$(q_0, w)$$

iar o configuratie finala este:

$$(q, \varepsilon), \text{ cu } q \in F.$$

Trecere

Un pas de functionare inseamna trecerea de la o configuratie la alta:

$$(q, a\alpha) \rightarrow (p, \alpha)$$

unde $p \in \delta(q, a)$.

Adica automatul citește simbolul a , trece din starea q în starea p , iar pe banda rămâne de citit α .

Limbaj acceptat

Limbajul acceptat de automatul finit A este mulțimea cuvintelor care duc automatul din configurația inițială într-o configurație finală:

$$L(A) = \{ w \in \Sigma^* \mid (q_0, w) \Rightarrow^* (p, \varepsilon), p \in F \}$$

Sursa în curs: Curs 1, I.2. Automate finite, pp. 12-14.

5. Automate pushdown - definiție, configurație, trecere, limbaj acceptat

Întrebare: Ce este un automat pushdown și ce limbaj acceptă?

Răspuns:

Automatele **pushdown** sunt mecanisme de recunoaștere a limbajelor independente de context. Numele vine de la organizarea memoriei auxiliare sub forma de **stivă**.

Un automat pushdown este un sistem:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

unde:

- Q este mulțimea stărilor automatului;
- Σ este alfabetul de intrare;
- Γ este alfabetul intern, adică alfabetul stivei pushdown;
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow P_f(Q \times \Gamma^*)$ este funcția de tranziție;
- q_0 este starea inițială;
- Z_0 este simbolul intern inițial al stivei;
- F este mulțimea stărilor finale.

Configurație

O configurație a automatului pushdown este un triplet:

$$(q, w, \gamma)$$

unde:

- q este starea curenta;
- w este portiunea din secventa de intrare ramasa de citit;
- γ este continutul memoriei pushdown, adica al stivei.

Configuratia initiala este:

$$(q_0, w, Z_0)$$

Trecere

Trecerea de la o configuratie la alta se face prin pasi. Daca:

$$(q_1, \alpha) \in \delta(q, a, Z)$$

atunci se poate trece din:

$$(q, aw, Z\gamma)$$

in:

$$(q_1, w, \alpha\gamma)$$

Adica automatul citeste simbolul a , scoate Z din varful stivei, introduce α si trece in starea q_1 .

Limbaj acceptat

In curs sunt prezentate doua criterii de acceptare:

1. Acceptare dupa criteriul benzii pushdown vide:

$$L_\varepsilon(P) = \{ w \in \Sigma^* \mid (q_0, w, Z_0) \Rightarrow^* (p, \varepsilon, \varepsilon) \}$$

Cuvantul este acceptat daca intrarea este consumata si stiva devine vida.

2. Acceptare dupa criteriul starii finale:

$$L(P) = \{ w \in \Sigma^* \mid (q_0, w, Z_0) \Rightarrow^* (p, \varepsilon, \alpha), p \in F \}$$

Cuvantul este acceptat daca intrarea este consumata si automatul ajunge intr-o stare finala.

Cele doua moduri de acceptare sunt echivalente: un limbaj este acceptat de un automat pushdown dupa criteriul starii finale daca si numai daca este acceptat dupa criteriul benzii pushdown vide.

Sursa in curs: Curs 3, I.6. Automate pushdown si limbaje independente de context, pp. 39-46.

6. Gramatici LL - definitie, PRIM, URM

Intrebare: Ce sunt gramaticile LL si cum sunt definite functiile PRIM si URM?

Raspuns:

Gramaticile **LL(k)** sunt gramatici independente de context pentru care derivarea la stanga poate fi determinata in mod unic folosind:

- partea deja citita din cuvantul de intrare;
- urmatoarele k simboluri de intrare;
- neterminalul care trebuie expandat.

Litera L din stanga indica faptul ca intrarea este parcursa de la **stanga la dreapta**, iar a doua litera L indica faptul ca se construieste o **derivare la stanga**.

Definitia functiei PRIM

Pentru o gramatica G , un numar natural k si un cuvant $\alpha \in (N \cup \Sigma)^*$, functia:

$PRIM_k(\alpha)$

contine prefixele terminale de lungime cel mult k ale cuvintelor care pot fi derivate din α .

Intuitiv, $PRIM_k(\alpha)$ raspunde la intrebarea:

Ce secvente terminale pot aparea primele daca derivam din α , privind cel mult k simboluri?

Pentru $k = 1$, aceasta functie corespunde ideii de **FIRST**.

Definitia gramaticii LL(k)

O gramatica $G = (N, \Sigma, P, S)$ este **LL(k)** daca, in orice situatie de derivare la stanga, urmatoarele k simboluri de intrare determina in mod unic productia care trebuie folosita pentru expandarea neterminalului curent.

Formal, daca exista doua derivari posibile pornind din aceeasi forma sententiala:

$S \Rightarrow^* wA\alpha \Rightarrow w\beta\alpha \Rightarrow^* wx$

si

$S \Rightarrow^* wA\alpha \Rightarrow w\gamma\alpha \Rightarrow^* wy$

iar:

$PRIM_k(x) = PRIM_k(y)$

atunci trebuie sa rezulte:

$$\beta = \gamma$$

Adica nu pot exista doua productii diferite pentru acelasi neterminal care sa produca aceeasi anticipare de lungime k .

Definitia functiei URM

Pentru $\beta \in (N \cup \Sigma)^*$, functia:

$$\text{URM}_k(\beta)$$

este definita ca multimea prefixelor de lungime cel mult k care pot urma dupa β intr-o forma sententiala derivata din simbolul initial.

Intuitiv, $\text{URM}_k(\beta)$ raspunde la intrebarea:

Ce secvente terminale pot aparea imediat dupa β intr-o derivare din S ?

Pentru $k = 1$, aceasta functie corespunde ideii de **FOLLOW**.

Sursa in curs: Curs 4, I.7.4. Gramatici LL(k), pp. 52-57.

7. Gramatici LR - definitie LR(k)

Intrebare: Ce este o gramatica LR(k)?

Raspuns:

Gramaticile **LR** au fost introduse de Knuth, iar clasa limbajelor generate de ele este clasa limbajelor independente de context deterministe. Analiza LR are avantajul generalitatii: limbajele de programare care admit o definitie sintactica in forma BNF sunt, in general, analizabile LR.

In analiza LR:

- intrarea este parcursa de la stanga la dreapta;
- se construiesc derivarea cea mai din dreapta, dar in sens invers;
- se folosesc k simboluri de anticipare pentru a decide reducerile.

Pentru construirea arborelui de derivare se folosesc urmatoarele informatii:

1. o pozitie curenta in cuvantul de analizat;
2. intregul context stanga al cuvantului sursa;

3. urmatoarele k simboluri ale sursei;
4. faptul ca, la fiecare pas, neterminalul care deriveaza este cel mai din dreapta.

O gramatica este **LR(k)** daca aceste informatii determina in mod unic:

- daca pozitia curenta indica limita dreapta a partii reductibile;
- limita din stanga a partii reductibile;
- productia folosita pentru reducere.

Pentru definitia formală, se considera gramatica extinsa G' , obtinuta prin adaugarea unui nou simbol initial S' si a productiei:

$$S' \rightarrow S$$

O gramatica G este **LR(k)** daca, in situatiile in care doua derivari la dreapta ar putea conduce la aceeasi anticipare de lungime k , partea reductibila si productia de reducere sunt determinate unic.

Sursa in curs: Curs 5, I.7.5. Gramatici LR(k), pp. 63-69.

8. Analiza sintactica - formularea problemei, principii descendente si ascendente

Intrebare: Cum este formulata problema analizei sintactice si care sunt principiile de functionare ale analizei descendente si ascendente?

Raspuns:

Formularea problemei

Fie o gramatica independenta de context:

$$G = (N, \Sigma, P, S)$$

si un cuvant:

$$\alpha \in L(G)$$

Spunem ca a fost efectuata **analiza sintactica** a secventei α daca a fost gasit cel putin un **arbore generator** pentru α .

Analiza sintactica poate fi:

- **la stanga**, cand se determina o derivare la stanga;

- **la dreapta**, cand se determina o derivare la dreapta.

Dispozitivele care furnizeaza analiza sintactica la stanga se numesc **analizoare descendente**.

Dispozitivele care furnizeaza analiza sintactica la dreapta se numesc **analizoare ascendente**.

Principiul analizei sintactice descendente

Analiza descendenta, numita si **top-down**, construiește arborele de derivare de la radacina spre frunze.

Principiul este urmatorul:

1. Se porneste de la simbolul initial S , ca radacina a arborelui.
2. Se alege o productie pentru neterminalul curent si se face o **expandare**.
3. Daca in varful stivei apare un terminal care coincide cu simbolul curent de intrare, terminalul este sters si citirea avanseaza cu o pozitie.
4. Daca alternativa aleasa nu corespunde sirului de intrare, analizorul revine si incearca alta alternativa.
5. Daca se consuma tot cuvantul de intrare si arborele este complet, analiza se termina cu succes.

In curs, algoritmul general de analiza descendenta foloseste backtracking si doua benzi pushdown: una pentru alternativele incercate si alta pentru simbolurile care trebuie inca analizate.

Principiul analizei sintactice ascendente

Analiza ascendenta, numita si **bottom-up**, construiește arborele de derivare pornind de la frunze spre radacina.

Principiul este urmatorul:

1. Se porneste de la sirul de intrare.
2. Se cauta o parte reductibila, adica o secventa care poate fi partea dreapta a unei productii.
3. Daca se gaseste o productie $A \rightarrow \beta$, secventa β este redusa la neterminalul A .
4. Reducerile continua pana cand se obtine simbolul initial S .
5. Daca o reducere aleasa duce la blocaj, analizorul revine si incearca alta reducere posibila.

Acceptarea are loc daca, dupa consumarea intrarii si aplicarea reducerilor, se ajunge la simbolul initial.

Sursa in curs: Curs 7, II.3.1-II.3.2, pp. 101-109.

9. Codul intermediar - generalitati

Intrebare: Ce este codul intermediar si care este rolul lui?

Raspuns:

Rezultatul analizei sintactice si semantice consta intr-un fisier care contine traducerea programului intr-un **limbaj intermediar**. Acesta este mai apropiat de limbajul de asamblare decat de limbajul sursa.

Programul intermediar este o succesiune de operatii impreuna cu operanzii lor. Operatiile sunt in mare parte similare celor din limbajul de asamblare:

- operatii aritmetice;
- atribuirii;
- teste;
- salturi.

Ordinea operatiilor in codul intermediar este ordinea in care acestea se executa.

Din codul intermediar lipsesc declaratiile; descrierea operanzilor se gaseste in **tabela de simboluri**.

Codul intermediar se deosebeste de limbajul de asamblare prin faptul ca operanzii nu sunt registri sau cuvinte de memorie, ci referinte la intrari in tabela de simboluri. Operanzii pot contine si informatii despre natura lor:

- variabile simple;
- variabile indexate;
- variabile temporare;
- constante;
- apeluri de functii;
- mod de adresare direct sau indirect.

Formele principale de reprezentare a codului intermediar prezentate in curs sunt:

1. **Forma poloneza** - notatie postfixata, utila pentru expresii aritmetice.
2. **Arbori sintactici** - reprezentare apropiata de structura sintactica a programului sursa.
3. **Cod cu trei adrese** - secventa de instructiuni simple, de forma $A := B \text{ op } C$.

Codul cu trei adrese poate fi implementat prin:

- **triplete**;
- **triplete indirecte**;

- **cuadrupele.**

Sursa in curs: Curs 10, Generarea codului intermediar, pp. 145-151.

10. Cod obiect - tipuri de cod obiect

Intrebare: Ce este codul obiect si ce tipuri de cod obiect sunt prezentate in curs?

Raspuns:

Ultima faza a compilarii are ca scop sinteza **programului obiect**, adica a unui program executabil sau aproape executabil. Acesta poate fi preluat de un alt procesor de limbaj implementat pe calculatorul tinta si tradus in cod executabil.

Generarea codului este una dintre cele mai importante si dificile faze ale compilarii, deoarece necesita cunostinte despre masina tinta si despre sistemul de operare.

Formele pe care le poate lua programul obiect sunt:

1. Program executabil

- Rezultatul este direct executabil.
- Este stocat intr-o zona de memorie fixa.
- Toate adresele sunt puse la zi.
- Este recomandat pentru programe mici.
- Dezavantaj: lipsa de flexibilitate; compilarea modulara este imposibila.

2. Program obiect / translatabil

- Inainte de executie necesita faza de editare de legaturi.
- Permite legarea rutinelor din biblioteci sau a rutinelor realizate de utilizatori.
- Este forma cea mai intalnita in compilatoarele comerciale.

3. Program in limbaj de asamblare

- Este cel mai simplu de generat.
- Instructiunile sunt apropiate de instructiunile cu trei adrese.
- Necesita o faza de asamblare inainte de executie.
- Rezultatul nu este direct utilizabil.

4. Program intr-un alt limbaj

- Simplifica generarea codului.
- Necesita cel putin o compilare suplimentara pentru a putea fi executat.
- Este cazul preprocesoarelor de limbaje.

Generarea codului obiect depinde de:

- calculatorul tinta;
- sistemul de operare;
- forma codului intermediar;
- optimizarile facute anterior;
- verificarile semantice deja realizate.

Sursa in curs: Curs 12, II.7.1. Generalitati, pp. 177-180.

11. Tabela de simboluri - generalitati privind structura ei

Intrebare: Ce este tabela de simboluri si ce structura are?

Raspuns:

Tabela de simboluri este structura in care compilatorul concentreaza informatia selectata despre numele simbolice care apar in programul sursa.

In forma cea mai simpla, tabela de simboluri este un tablou de inregistrari. Ea poate fi reprezentata si prin alte structuri, cum ar fi:

- arbori;
- liste;
- tabele dispersate.

Tabela de simboluri este divizata in doua parti:

1. Numele

- este un sir de caractere care reprezinta identificatorul.

2. Informatia asociata numelui

- contine attributele identificatorului, cum ar fi:
 - tipul: `integer`, `real`, `record`, `array` etc.;
 - utilizarea: parametru formal, eticheta, subprogram etc.;
 - domeniul de definitie: numar de dimensiuni, limite pentru tablouri etc.;
 - adresa de memorie.

Divizarea se face pentru economie de memorie. Numele simbolice pot deveni inutile dupa iesirea din domeniul lor de valabilitate, iar spatiul ocupat de ele poate fi reutilizat.

Utilizarea tabelii de simboluri in compilare

Tabela de simboluri este folosita in mai multe faze:

- **Analiza lexicala si sintactica** - se verifica daca un nume este deja memorat; daca nu, este introdus in tabela.
- **Analiza semantica** - se verifica daca identificatorii sunt utilizati conform declaratiilor.
- **Generarea codului** - se determina lungimea zonelor de memorie alocate variabilelor.
- **Tratarea erorilor** - se evita mesajele de eroare redundante.

Actiuni asupra tabelii de simboluri

Compilerul executa asupra tabelii de simboluri operatii precum:

- verificarea daca un nume apare pentru prima data;
- adaugarea unui nume nou;
- adaugarea de informatii la un nume existent;
- stergerea unui nume sau a unui grup de nume.

Organizarea tabelii

Performanta compilerului depinde mult de eficienta cautarii in tabela de simboluri, deoarece cautarea se face pentru fiecare simbol intalnit. Din acest motiv, metoda de organizare este foarte importanta.

In curs sunt prezentate urmatoarele forme de organizare:

1. **Tabele neordonate** - intrarile sunt adaugate secvential, in ordinea aparitiei.
2. **Tabele ordonate alfabetic** - permit cautare binara.
3. **Tabele arborescente** - folosesc arbori binari pentru reprezentarea simbolurilor.
4. **Tabele dispersate** - folosesc functii de dispersie pentru acces rapid; coliziunile pot fi tratate prin inlantuire.

Sursa in curs: Curs 13, II.8.1-II.8.2, pp. 193-203.

Recapitulare rapida

- **Compilerul** traduce programul sursa in program obiect si parcurge faze precum analiza lexicala, sintactica, semantica, cod intermediar, optimizare si cod obiect.
- **Gramatica** este $G = (N, \Sigma, P, S)$, iar limbajul generat este $L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$.
- **Clasificarea Chomsky:** tip 0, tip 1, tip 2, tip 3, cu incluziunea $L_3 \subset L_2 \subset L_1 \subset L_0$.
- **Limbaj recursiv:** exista algoritm care decide apartenenta oricarui cuvant la limbaj.
- **Automatul finit** recunoaste limbaje regulate.

- **Automatul pushdown** recunoaste limbaje independente de context si foloseste o stiva.
- **Gramaticile LL(k)** permit analiza determinista descendenta cu k simboluri de anticipare.
- **Gramaticile LR(k)** permit analiza determinista ascendenta cu k simboluri de anticipare.
- **Analiza sintactica** inseamna gasirea unui arbore generator pentru cuvantul analizat.
- **Codul intermediar** este o reprezentare intre programul sursa si codul obiect.
- **Codul obiect** poate fi executabil, translatabil, in limbaj de asamblare sau intr-un alt limbaj.
- **Tabela de simboluri** retine identificatorii si attributele lor si este folosita in mai multe faze ale compilarii.